Министерство науки и высшего образования Российской Федерации

Федеральное государственное автономное образовательное
учреждение высшего образования
Национальный исследовательский Нижегородский государственный
университет им. Н.И. Лобачевского

Институт информационных технологий, математики и механики

Отчет по лабораторной работе №1

«Множества на основе битовых полей»

Выполнил:
студент группы 3825Б1ФИсп1
Лаптев М.Е.

Проверил:
Ивлев А.Д.

Нижний Новгород
2026

Содержание

Введение 3
1. Постановка задачи 4
2. Руководство пользователя 5
3. Руководство программиста 6
3.1. Описание структуры программы 6
3.2. Описание структур данных 6
3.3. Описание алгоритмов 7
4. Результаты экспериментов и тестирования 8
Заключение 8
Литература 10
Приложение 11

Введение

Целью данной лабораторной работы является изучение принципов объектно-ориентированного программирования на языке C++, освоение работы с динамической памятью, а также приобретение навыков разработки автоматических тестов с использованием фреймворка Google Test. В ходе работы реализуются структуры данных для битовых полей (TBitField) и множеств на основе битовых полей (TSet).

Битовое поле — это структура данных, представляющая собой массив битов фиксированной длины. Оно позволяет компактно хранить последовательность нулей и единиц, экономя память по сравнению с массивом логических переменных. В данной работе битовое поле реализуется через массив целых чисел (unsigned int), где каждый бит числа соответствует одному элементу поля, и поддерживает базовые побитовые операции (И, ИЛИ, НЕ, сдвиг) и доступ к отдельным битам по индексу.

Множество (TSet) — это математический объект, представляющий собой набор уникальных элементов, порядок которых не важен. В рамках лабораторной работы множество реализуется с помощью композиции: внутри класса TSet используется битовое поле (TBitField) в качестве характеристического вектора (где 1 означает наличие элемента, 0 - его отсутствие). Это позволяет эффективно выполнять теоретико-множественные операции (объединение, пересечение, дополнение) через побитовые операции над внутренним полем, не наследуя при этом методы, характерные для битовой строки.

1. Постановка задачи

В соответствии с заданием необходимо разработать два класса:

1. TBitField — класс, реализующий битовое поле (массив битов фиксированной длины). Класс должен поддерживать установку, сброс и чтение бита, получения длинны строки, различные операторы перегрузки, также операторы ввода и вывода в поток, а также конструктор копирования и деструктор.
2. TSet — класс, реализующий математическое множество на основе битового поля (композиция). Класс должен поддерживать теоретико-множественный операции, операции сравнения, операции ввода и вывода. Методы: GetMaxPower, InsElem, DelElem, IsMember. Различные конструкторы по мощности, копирования, преобразования из битового поля. И оператор преобразования из битового поля.

Конкретно все название методов находятся в папке includes/Lab1. Их реализацию мы выполнить в своих файлах.

Дополнительно требуется написать тесты с использованием фреймворка Google Test для проверки корректности работы всех методов классов.

2. Руководство пользователя

Программа представляет собой набор автоматических тестов, не требующих интерактивного ввода данных пользователем во время выполнения. Для запуска программы необходимо:

1. Открыть решение в среде Microsoft Visual Studio.
2. Собрать проект (Сборка -> Собрать решение).
3. Открыть окно «Обозреватель тестов» (Тест -> Обозреватель тестов).
4. Нажать кнопку «Запустить все» для прогона тестов. Результаты отображаются в виде зеленых галочек (тест пройден) или красных крестиков (тест не пройден).

Тесты прописаны таким образом, что взаимодействуют со всеми методами, конструкторами, операторами, перегрузками операций. Таким образом для проверки работы программы необходимо проверить только работу тестов.

3. Руководство программиста

3.1. Описание структуры программы

Программа состоит из двух логических модулей (проектов), объединенных в одно решение (Solution):

1. **Модуль** Lab1 — является основным модулем, содержащим реализацию структур данных. В его состав входят:
   - Файлы заголовков (tbitfield.h, tset.h), подключенные из внешней папки includes. Они содержат объявления классов.
   - Файл tbitfield.cpp — содержит реализацию методов класса битового поля TBitField.
   - Файл tset.cpp — содержит реализацию методов класса множества TSet.

   Назначение модуля: предоставление готовых инструментов (классов) для работы с битовыми полями и множествами.
2. **Модуль** Sample-Test1 — модуль, генерируемый фреймворком Google Test. В его состав входят:
   - Файл test.cpp — содержит набор тестовых сценариев (TEST-блоков).
   - Файлы pch.h и pch.cpp — для предварительно скомпилированных заголовков.

   Назначение модуля: автоматическая проверка корректности работы классов из модуля Lab1. Модуль ссылается на Lab1 для получения доступа к реализациям.

3.2. Описание структур данных

В программе реализованы два основных пользовательских класса:

- **Класс** TBitField **(Битовое поле)**
  - *Назначение:* хранение и обработка последовательности битов фиксированной длины.
  - *Поля класса:*
    - int BitLen — длина битового поля в битах (максимальное количество битов).
    - TELEM *pMem — указатель на динамический массив элементов типа unsigned int (где TELEM — псевдоним для unsigned int), в котором физически хранятся биты.
    - int MemLen — количество элементов в массиве pMem, необходимое для хранения BitLen битов.
  - *Методы:* конструкторы (по умолчанию и копирования), деструктор для очистки памяти, методы доступа к битам (SetBit, ClrBit, GetBit), перегруженные операторы побитовых операций (|, &, ~), сравнения (==, !=), присваивания и ввода/вывода.
- **Класс** TSet **(Множество)**
  - *Назначение:* реализация математического множества уникальных целых чисел.
  - *Поля класса:*

- int MaxPower — максимальная мощность множества (максимально возможное количество элементов).
- TBitField BitField — объект битового поля, используемый в качестве характеристического вектора (композиция). Если i-й бит равен 1, значит, элемент i присутствует во множестве.

- *Методы:* методы для добавления (InsElem) и удаления (DelElem) элементов, проверки принадлежности (IsMember), а также перегруженные операторы для выполнения теоретико-множественных операций (объединение +, пересечение *, дополнение ~), сравнения и ввода/вывода.

3.3. Описание алгоритмов

В основе работы программы лежат следующие алгоритмы:

1. **Алгоритм доступа к битам (в классе** TBitField**):** Поскольку память выделяется под массив 32-битных чисел, для доступа к биту с номером n вычисляется индекс элемента в массиве как n / 32, а позиция бита внутри этого элемента — как n % 32. Для установки бита используется побитовая операция ИЛИ с маской (1 << позиция). Для сброса — побитовое И с инвертированной маской. Для проверки — побитовое И с маской.

2. **Алгоритм копирования (Конструктор копирования):** При создании копии объекта TBitField (или TSet) выделяется новая область памяти (new TELEM[MemLen]), после чего в цикле поэлементно копируются данные из исходного массива в новый.

3. **Алгоритмы теоретико-множественных операций (в классе** TSet**):** Благодаря композиции, операции над множествами сводятся к побитовым операциям над внутренним полем BitField:

   - *Объединение (+):* выполняется побитовое ИЛИ (|) характеристических векторов.

   - *Пересечение (*):* выполняется побитовое И (&) характеристических векторов.

   - *Дополнение (~):* выполняется побитовое НЕ (~) над вектором с последующей очисткой неиспользуемых битов.

4. **Алгоритм ввода/вывода множества:** При выводе (operator<<) программа проходит циклом от 0 до MaxPower. Если бит с текущим индексом установлен в 1 (элемент принадлежит множеству), его номер выводится в поток. При вводе (operator>>) сначала считывается количество элементов, затем в цикле считываются сами числа и добавляются во множество через метод InsElem

4. Результаты экспериментов и тестирования

Для проверки работоспособности классов было разработано 11 тестов с использованием фреймворка Google Test. Тесты покрывают следующие сценарии:

- Корректность создания полей и множеств заданной размерности.
- Установка, сброс и чтение отдельных битов.
- Логические операции над битовыми полями (|, &, ~).
- Операции добавления и удаления элементов множества.
- Теоретико-множественные операции (объединение +, пересечение *, дополнение ~).
- Проверка операторов сравнения (==, !=).
- Проверка корректности ввода и вывода.

По результатам запуска все тесты были успешно пройдены (11 из 11). Это подтверждает корректность реализованных алгоритмов и отсутствие утечек памяти (благодаря корректно написанным конструкторам копирования и деструкторам).

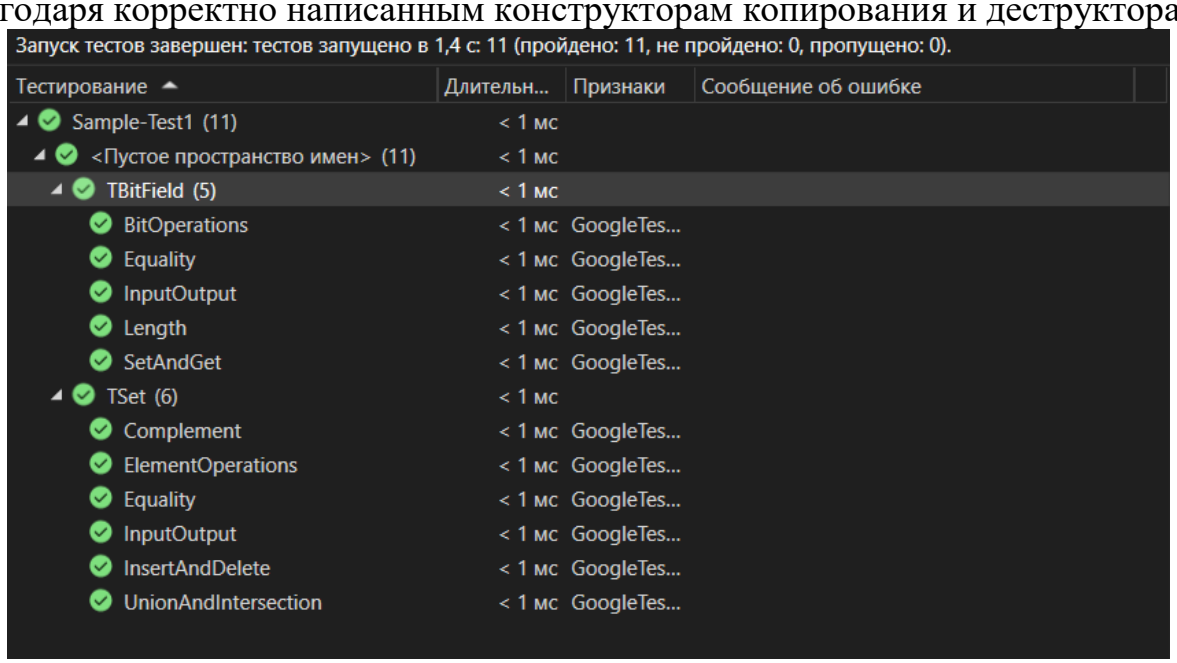


рис 1. Результаты тестов

Заключение

В ходе выполнения лабораторной работы были получены практические навыки работы с динамической памятью в C++, изучены принципы композиции классов (на примере использования битового поля внутри множества) и освоен инструмент модульного тестирования Google Test. Все поставленные задачи были выполнены в полном объеме, программа работает корректно и проходит все тестовые сценарии.

Литература

1. Страуструп Бьерн Язык программирования C++. Специальное издание. Пер. с англ. – М.: Издательство Бином, 2017 г. – 1136 с.
2. Сайт cppreference: https://en.cppreference.com/
3. Гергель В.П., Сысоев А.В. Методы программирования. – ННГУ, 2015.
4. Документация Google Test. URL: https://google.github.io/googletest/

Приложение